



Practica para la materia:

Programación orientada a objetos

Presentada por:

Arteaga Castañeda Luis Eduardo

Matricula:242160002

4to cuatrimestre

Carrera:

***Ingeniería en Sistemas
Computacionales***



1. Introducción

Este documento tiene como objetivo proporcionar una explicación técnica y detallada sobre el funcionamiento de un programa en C#, desarrollado como parte de la práctica correspondiente a la materia de Programación Orientada a Objetos (POO). El programa implementa una versión simplificada del clásico juego "Buscaminas", con un enfoque en el desarrollo de lógica de juego, estructuras de control, validación de entradas y manipulación de arreglos.

El código permite al jugador avanzar por niveles, donde deberá descubrir casillas seguras dentro de un campo lineal, evitando pisar minas colocadas aleatoriamente. A medida que se incrementa el nivel, el número de minas también aumenta, lo que añade dificultad progresiva.

Las funcionalidades principales del programa incluyen:

- Generación aleatoria de minas dentro de un campo de juego.
- Presentación visual del campo para el usuario.
- Validación de entradas numéricas por parte del usuario.
- Cálculo de minas adyacentes para ayudar al jugador.
- Gestión de vidas y progreso por niveles.

El propósito del programa no es solamente brindar una experiencia de juego interactiva, sino también reforzar los conceptos de programación orientada a objetos y lógica algorítmica mediante una aplicación práctica y entretenida.

2. Descripción general del código

El programa fue desarrollado en el lenguaje de programación C# y tiene como objetivo implementar un juego lineal de tipo Buscaminas, en el cual el jugador debe descubrir casillas seguras dentro de un campo unidimensional sin pisar minas ocultas.

La lógica del juego está estructurada en niveles, y en cada uno de ellos se incrementa progresivamente la cantidad de minas. El campo de juego es de longitud fija (10 casillas), pero la cantidad de minas varía según el nivel actual. El jugador tiene cinco vidas por nivel y debe encontrar todas las casillas seguras para avanzar.

El código implementa las siguientes estructuras y funcionalidades:

- **Arreglos unidimensionales:** Uno para representar el campo visible del jugador (`char[]`) y otro para el campo real con las minas (`int[]`).

- **Ciclos for y while:** Se utilizan para la inicialización del campo, el control de juego por turnos, y la progresión por niveles.
- **Validación de entradas:** Se asegura que el usuario introduzca únicamente números válidos dentro del rango del campo.
- **Generador aleatorio:** Se usa Random para ubicar minas de forma aleatoria sin repetir posiciones.
- **Lógica de juego:** Determina el comportamiento al descubrir una casilla (mina o segura), y muestra la cantidad de minas adyacentes si es una casilla segura.
- **Control de flujo:** Permite reiniciar el juego si el jugador pierde todas sus vidas, o continuar al siguiente nivel si supera el actual.

Esta estructura proporciona un ejemplo práctico del uso de programación estructurada y control de estado en aplicaciones interactivas, siendo útil para el aprendizaje de la lógica básica en juegos por consola.

3. Explicación de código

- *Inicialización del campo*

Al comenzar el programa, se definen las constantes necesarias para establecer el tamaño del campo de juego (`iLongitudCampo = 10`) y la cantidad total de niveles (`iNivelesTotales = 5`). Además, se declara un objeto Random para la colocación aleatoria de minas.

```

3      const int iLongitudCampo = 10; // Longitud del campo de juego (10 casillas)
4      const int iNivelesTotales = 5; // Número total de niveles del juego
5      Random random = new Random(); // Generador de números aleatorios para ubicar minas
6
7      // Ciclo principal por niveles
8      for (int iNivel = 1; iNivel <= iNivelesTotales; iNivel++)
9      {
10         Console.Clear();
11         Console.WriteLine($"Nivel {iNivel}: encuentra todas las casillas seguras (hay {iNivel} mina(s))");
12
13         // Campo visible para el jugador
14         char[] cCampoVisible = new char[iLongitudCampo];
15
16         // Campo real con minas (0 = seguro, 1 = mina)
17         int[] iCampoReal = new int[iLongitudCampo];
18
19         // Inicializar campo visible con caracteres '#'
20         for (int i = 0; i < iLongitudCampo; i++)
21         {
22             cCampoVisible[i] = '#';
23         }
24     }

```

Dentro del ciclo principal que recorre los niveles del juego (`for (int iNivel = 1; iNivel <= iNivelesTotales; iNivel++)`), se realiza la inicialización del campo para ese nivel. Se utilizan dos arreglos unidimensionales:

- `char[] cCampoVisible`: Representa el campo que ve el jugador. Inicialmente todas sus posiciones contienen el carácter '#', que indica una casilla no descubierta.

- `int[] iCampoReal`: Representa el campo real donde se colocan las minas. Se inicializa con ceros (0), donde 0 indica una casilla segura y 1 una mina.

Este diseño permite separar la lógica interna del juego (campo real con minas) de la visualización que se presenta al jugador (campo visible), facilitando así la interacción y ocultando las minas hasta que se descubren.

- **Colocación de minas**

Una vez inicializado el campo, el siguiente paso es colocar las minas de forma aleatoria. La cantidad de minas por nivel es igual al número del nivel actual (`iNivel`), lo que significa que la dificultad del juego aumenta progresivamente conforme se avanza.

Para colocar las minas, se utiliza un ciclo `while` que se ejecuta hasta que el número de minas colocadas (`iMinasColocadas`) sea igual al número de minas requeridas en ese nivel. Dentro del ciclo:

- Se genera una posición aleatoria usando `random.Next(0, iLongitudCampo)`.
- Se verifica si esa posición aún no contiene una mina (`iCampoReal[iPos] == 0`).
- Si está libre, se asigna un 1 en esa posición del arreglo `iCampoReal` para marcar la presencia de una mina y se incrementa el contador de minas colocadas.

Este enfoque garantiza que las minas se ubiquen en posiciones únicas sin sobrescribir otras minas ya colocadas, conservando la aleatoriedad y la integridad del juego.

```
// Colocar minas aleatoriamente en el campo real
int iMinasColocadas = 0;
while (iMinasColocadas < iNivel)
{
    int iPos = random.Next(0, iLongitudCampo); // Posición aleatoria
    if (iCampoReal[iPos] == 0) // Solo colocar si está libre
    {
        iCampoReal[iPos] = 1; // Coloca una mina
        iMinasColocadas++;
    }
}

int iCasillasSeguras = iLongitudCampo - iNivel; // Total de casillas sin mina
int iDescubiertas = 0; // Contador de casillas descubiertas
int iVidas = 5; // Vidas por nivel
bool bPerdiste = false; // Manera para saber si el jugador perdió
```

- **Turno del jugador**

Una vez que las minas han sido colocadas, inicia el bucle del turno del jugador. Este ciclo se mantiene activo mientras el jugador no haya perdido todas sus vidas (`iVidas > 0`) y no haya descubierto todas las casillas seguras (`iDescubiertas < iCasillasSeguras`).

Dentro del ciclo se desarrollan las siguientes acciones:

- **Visualización del campo:** Se muestra el campo visible (`cCampoVisible`) en la consola, representando con # las casillas no descubiertas, 0 o 1 las casillas seguras (con minas adyacentes o no) y * las casillas donde se ha pisado una mina.
- **Entrada del jugador:** Se solicita una posición del 1 al 10. Se valida que la entrada sea numérica y que esté dentro del rango permitido. Si la entrada es inválida, se muestra un mensaje de error y se repite la solicitud.
- **Validación de casillas:** Si el jugador intenta descubrir una casilla que ya ha sido revelada (es decir, su valor ya no es #), se le avisa que esa posición ya fue descubierta.
- **Evaluación de la jugada:**
 - Si la casilla contiene una mina (`iCampoReal[iIndice] == 1`), se muestra un * en esa posición, se reduce una vida, y se indica que el jugador ha pisado una mina. Si las vidas llegan a cero, el juego termina.
 - Si la casilla es segura, se calcula cuántas minas hay en las posiciones adyacentes (izquierda y derecha). Se muestra 1 si hay al menos una mina cercana, o 0 si no hay minas adyacentes. Esta casilla se marca como descubierta y se incrementa el contador de casillas seguras descubiertas.

```
// Bucle del turno del jugador
while (!bPerdiste && iDescubiertas < iCasillasSeguras)
{
    // Mostrar campo visible al jugador
    foreach (char c in cCampoVisible)
        Console.Write(c + " ");
    Console.WriteLine($"{iVidas} restantes: {iVidas}");

    // Solicitar posición al usuario
    Console.Write($"Ingresa una posición (1 - {iLongitudCampo}): ");
    int iPosicion;
    if (!int.TryParse(Console.ReadLine(), out iPosicion) || iPosicion < 1 || iPosicion > iLongitudCampo)
    {
        Console.WriteLine("Entrada inválida. Intenta de nuevo.");
        continue;
    }

    int iIndice = iPosicion - 1; // Convertir a índice de arreglo

    // Verificar si la casilla ya fue descubierta
    if (cCampoVisible[iIndice] != '#')
    {
        Console.WriteLine("Ya descubriste esta posición.");
        continue;
    }
}
```

```

// Verificar si hay mina en la posición
if (iCampoReal[iIndice] == 1)
{
    cCampoVisible[iIndice] = '*'; // Mostrar mina
    iVidas--; // Pierde una vida
    Console.WriteLine("Pisaste una mina!");

    if (iVidas == 0)
    {
        Console.WriteLine("Te quedaste sin vidas. Fin del juego.");
        bPerdiste = true; // Terminar juego
    }
}
else
{
    // Contar minas adyacentes a izquierda y derecha
    int iMinasadyacentes = 0;
    if (iIndice > 0 && iCampoReal[iIndice - 1] == 1) iMinasadyacentes++;
    if (iIndice < iLongitudCampo - 1 && iCampoReal[iIndice + 1] == 1) iMinasadyacentes++;

    // Mostrar '1' si hay minas cerca, '0' si no
    cCampoVisible[iIndice] = iMinasadyacentes > 0 ? '1' : '0';
    iDescubiertas++; // Incrementa casillas seguras descubiertas
}
}

```

- ***Progresión de nivel y finalización del juego***

Una vez que el jugador ha descubierto todas las casillas seguras en un nivel, se considera que ha superado ese nivel. En ese caso, se muestra un mensaje de felicitación, junto con el estado final del campo visible.

Si el jugador aún no ha alcanzado el último nivel, se le solicita que presione Enter para continuar. El juego limpia la consola y repite el proceso en el siguiente nivel, ahora con una mina adicional, aumentando así la dificultad.

En cambio, si el jugador pierde todas sus vidas en cualquier nivel, se muestra un mensaje indicando que el juego ha terminado y se rompe el ciclo principal de niveles, finalizando la ejecución del programa.

Finalmente, después de que el jugador complete todos los niveles con éxito o pierda, el programa muestra un mensaje de agradecimiento por haber jugado.

Esta lógica de progresión y finalización controla el flujo general del juego y asegura que cada partida tenga un comienzo, una evolución basada en el rendimiento del jugador y una conclusión clara, ya sea por victoria o derrota.

4. Pruebas del programa

```
Nivel 1: encuentra todas las casillas seguras (hay 1 mina(s))
# # # # # # # # # #
Vidas restantes: 5
Ingresa una posición (1 - 10): 1
0 # # # # # # # # #
Vidas restantes: 5
Ingresa una posición (1 - 10): 2
0 0 # # # # # # # #
Vidas restantes: 5
Ingresa una posición (1 - 10): 3
0 0 0 # # # # # # #
Vidas restantes: 5
Ingresa una posición (1 - 10): 4
0 0 0 0 # # # # # #
Vidas restantes: 5
Ingresa una posición (1 - 10): 5
0 0 0 0 0 # # # # #
Vidas restantes: 5
Ingresa una posición (1 - 10): 6
0 0 0 0 0 1 # # # #
Vidas restantes: 5
Ingresa una posición (1 - 10): |
```

Perder vidas:

```
Ingresa una posición (1 - 10): 7
¡Pisaste una mina!
0 0 0 0 0 1 * # # #
Vidas restantes: 4
Ingresa una posición (1 - 10): |
```

Colocar una casilla ya descubierta:

```
Ingresa una posición (1 - 10): 3
Ya descubriste esta posición.
0 0 0 0 0 1 * # # #
Vidas restantes: 4
Ingresa una posición (1 - 10): |
```

Pasar de nivel:

```
Ingresa una posición (1 - 10): 10

Resultado del nivel:
0 0 0 0 0 1 * 1 0 0
¡Nivel superado!
Presiona Enter para continuar al siguiente nivel...
|
```

Nivel 2:

```
Ingresa una posición (1 - 10): 1
0 # # # # # # # # # #
Vidas restantes: 5
Ingresa una posición (1 - 10): 2
0 1 # # # # # # # # # #
Vidas restantes: 5
Ingresa una posición (1 - 10): 4
0 1 # 1 # # # # # # # # # #
Vidas restantes: 5
Ingresa una posición (1 - 10): 5
0 1 # 1 0 # # # # # # # # # #
Vidas restantes: 5
Ingresa una posición (1 - 10): 6
0 1 # 1 0 1 # # # # # # # # # #
Vidas restantes: 5
Ingresa una posición (1 - 10): 8
0 1 # 1 0 1 # 1 # # # # # # # # # #
Vidas restantes: 5
Ingresa una posición (1 - 10): 9
0 1 # 1 0 1 # 1 0 # # # # # # # # # #
Vidas restantes: 5
Ingresa una posición (1 - 10): 10

Resultado del nivel:
0 1 # 1 0 1 # 1 0 0
¡Nivel superado!
Presiona Enter para continuar al siguiente nivel...
```


Perder partida:

```
Ingresa una posición (1 - 10): 7
¡Pisaste una mina!
Te quedaste sin vidas. Fin del juego.
```

```
Resultado del nivel:
```

```
0 0 0 0 0 1 * # # #
```

```
Perdiste. ¡Intenta otra vez desde el nivel 1!
```

```
¡Gracias por jugar!
```